DESARROLLO DE APLICACIONES CON OBJETOS · 1º PARCIAL · TEMA 1

Programación estructurada y funcional en Python

Los 26 ejercicios del repositorio de la cátedra con el enunciado tal cual, la solución probada y la salida real

26 ejercicios | Archivos, conjuntos y diccionarios | Programación funcional | Errores de la cátedra corregidos

Cómo usar esta guía

Los **26 ejercicios del tema 1** del repositorio de la cátedra (2026-guias), en el mismo orden, con el **enunciado tal cual**, la solución y la **salida real** con los archivos de datos de la cátedra. Todo el código y los datos están en el ZIP, una carpeta por ejercicio, para correrlos en VS Code. Probado con Python 3.11, sin librerías extra.

Qué significa cada etiqueta

- **Cátedra** la solución publicada por la cátedra, tal cual.
- **Corregida** la de la cátedra tenía un error: se arregló y se explica cuál era.
- **Reescrita** la de la cátedra anda; se reescribió más corta o sin la librería `rich`.
- **Propuesta** la cátedra no publicó solución.

	Ejercicio	Solución
Repaso de programación estructurada		
01	Hola mundo	Cátedra
02	Estación de servicio	Corregida
03	Ruleta	Corregida
04	Temperaturas	Corregida
Archivos		
05	Personas (versión 1, sin estructuras de datos)	Reescrita
06	Archivo de números	Reescrita
07	Archivo de códigos postales (CSV)	Reescrita
08	Análisis de propinas	Reescrita
09	Archivo de códigos postales (JSON)	Reescrita
10	JSON multinivel	Reescrita
11	Palabras del Quijote (archivos y conjuntos)	Corregida
Conjuntos		
12	Eliminación de repetidos	Cátedra
13	Números con un conjunto	Corregida
14	Operaciones de conjuntos	Propuesta
Diccionarios		
15	Agenda de países	Cátedra
16	Calculadora con diccionario de operaciones	Cátedra
17	Padrón de personas (versión 2, con diccionario)	Reescrita
18	Conteo de códigos postales por provincia	Cátedra
Programación funcional		
19	Calculadora	Corregida
20	Tito el robot	Corregida
21	Lista de números	Corregida
22	Personas (versión 3, funcional)	Corregida
23	Map/filter con una lista de números	Propuesta
24	Map/filter/reduce con una lista de palabras	Propuesta
25	Ejemplos: funciones como argumento	Cátedra
26	Ejemplos: lambda y comprensión de listas	Corregida

Repaso de programación estructurada

1 Hola mundo

Solución de la cátedra

Enunciado (tal cual)

Desarrollar un programa que muestre un mensaje por la pantalla. La presentación del mensaje debe realizarse en una función programada en un módulo.

saludos.py

```
def saludar():
    print("Hola mundo!!!")
```

principal.py

```
from saludos import *

saludar()
```

Salida al correrlo

```
Hola mundo!!!
```

2 Estación de servicio

Solución de la cátedra, corregida

Enunciado (tal cual)

Una estación de servicio que dispone de 10 surtidores y necesita gestionar información relacionada con la venta de combustibles en la jornada.

De cada surtidor se conoce:

- Número de Surtidor (validar que sea un número entre 1 y 30)
- Cantidad: representa la cantidad de litros de combustible vendido por el surtidor (validar que sea un número positivo)
- Tipo: representa el tipo de combustible del surtidor. Los valores que puede asumir son 1 representa “Nafta Super", 2 representa “Nafta Especial” y 3 representa “Gas Oil” (validar que se ingresen valores vválidos).

Se pide calcular e imprimir:

- El total de litros vendidos en la jornada, por tipo de combustible.
- El número de surtidor que menos combustible vendió.
- El promedio por surtidor en litros de combustible vendido en la jornada (promedio general, es un único resultado).

Qué se corrigió

- El promedio se calculaba con `//` (división entera) y se perdían los decimales.
- El menor se buscaba con `if not menor`: si un valor era 0 lo tomaba como “no hay menor todavía”. Acá se usa `None`.

principal.py

```
TIPOS = {1: "Nafta Super", 2: "Nafta Especial", 3: "Gas Oil"}
SURTIDORES = 10


def ingresar_entero(mensaje, minimo, maximo=None):
    """Pide un entero hasta que esté en el rango [minimo, maximo]."""
    valor = int(input(mensaje))
    while valor < minimo or (maximo is not None and valor > maximo):
        print("Valor inválido")
        valor = int(input(mensaje))
    return valor


def main():
    litros_por_tipo = {1: 0, 2: 0, 3: 0}
    menor_litros = None
    surtidor_menor = None
    total = 0

    for _ in range(SURTIDORES):
        numero = ingresar_entero("Número de surtidor (1 a 30): ", 1, 30)
        litros = ingresar_entero("Litros vendidos: ", 0)
        tipo = ingresar_entero("Tipo de combustible (1 a 3): ", 1, 3)
        litros_por_tipo[tipo] += litros
        total += litros
        if menor_litros is None or litros < menor_litros:      # "is None": un surtidor con 0
litros también cuenta
            menor_litros, surtidor_menor = litros, numero

    for tipo, nombre in TIPOS.items():
        print(f"Litros de {nombre}: {litros_por_tipo[tipo]}")
    print(f"Surtidor que menos vendió: {surtidor_menor} ({menor_litros} litros)")
    print(f"Promedio por surtidor: {total / SURTIDORES:.2f} litros")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Número de surtidor (1 a 30): 12
Litros vendidos: 500
Tipo de combustible (1 a 3): 1
Número de surtidor (1 a 30): 5
Litros vendidos: 300
Tipo de combustible (1 a 3): 2
Número de surtidor (1 a 30): 40
Valor inválido
Número de surtidor (1 a 30): 7
Litros vendidos: 0
Tipo de combustible (1 a 3): 3
Número de surtidor (1 a 30): 3
Litros vendidos: 450
Tipo de combustible (1 a 3): 3
Número de surtidor (1 a 30): 8
Litros vendidos: 1200
Tipo de combustible (1 a 3): 1
Número de surtidor (1 a 30): 15
Litros vendidos: 800
Tipo de combustible (1 a 3): 5
Valor inválido
Tipo de combustible (1 a 3): 2
Número de surtidor (1 a 30): 21
Litros vendidos: 650
Tipo de combustible (1 a 3): 1
Número de surtidor (1 a 30): 9
Litros vendidos: 900
Tipo de combustible (1 a 3): 3
Número de surtidor (1 a 30): 30
Litros vendidos: 150
Tipo de combustible (1 a 3): 2
Número de surtidor (1 a 30): 1
Litros vendidos: 380
Tipo de combustible (1 a 3): 1
Litros de Nafta Super: 2730
Litros de Nafta Especial: 1250
Litros de Gas Oil: 1350
Surtidor que menos vendió: 7 (0 litros)
Promedio por surtidor: 533.00 litros
```

3 Ruleta

Solución de la cátedra, corregida

Enunciado (tal cual)

Desarrollar un programa que simule el juego de la ruleta.

Para ello generar al azar 1000 tiradas y luego informar:

- Cantidad de pares e impares
- Cantidad de tiradas por cada docena
- Porcentaje de ceros sobre el total de jugadas.
- Cantidad de rojos y de negros

Qué se corrigió

- La docena se calculaba con `nro // 12`: del 1 al 11 caían en la 3ª docena, del 12 al 23 en la 1ª y del 24 al 35 en la 2ª. La cuenta correcta es `(nro - 1) // 12`.
- El 0 se contaba como par y dentro de la 3ª docena: en la ruleta el 0 no es par, ni impar, ni de ninguna docena.

principal.py

```python
import random

random.seed(2026)        # para que la salida sea siempre la misma (sacarlo para que cambie)
TIRADAS = 1000


def reducir(numero):
    """Suma los dígitos hasta que quede uno solo (ej.: 29 -> 11 -> 2)."""
    while numero >= 10:
        numero = sum(int(d) for d in str(numero))
    return numero


def color(numero):
    """V (verde) el 0; N (negro) el 10, el 28 y los que se reducen a par; R (rojo) el resto."""
    if numero == 0:
        return "V"
    if numero in (10, 28) or reducir(numero) % 2 == 0:
        return "N"
    return "R"


def main():
    tiradas = [random.randint(0, 36) for _ in range(TIRADAS)]
    sin_cero = [t for t in tiradas if t != 0]            # el 0 no es par, ni impar, ni de
ninguna docena
    pares = len([t for t in sin_cero if t % 2 == 0])
    docenas = [0, 0, 0]
    for t in sin_cero:
        docenas[(t - 1) // 12] += 1                      # 1-12 -> 0, 13-24 -> 1, 25-36 -> 2
    colores = [color(t) for t in tiradas]

    print(f"Pares: {pares} · Impares: {len(sin_cero) - pares}")
    for i, cantidad in enumerate(docenas):
        print(f"{i + 1}ª docena ({i * 12 + 1} a {i * 12 + 12}): {cantidad}")
    print(f"Ceros: {tiradas.count(0) / TIRADAS * 100:.1f} % de las tiradas")
    print(f"Rojos: {colores.count('R')} · Negros: {colores.count('N')}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Pares: 481 · Impares: 495
1ª docena (1 a 12): 313
2ª docena (13 a 24): 307
3ª docena (25 a 36): 356
Ceros: 2.4 % de las tiradas
Rojos: 474 · Negros: 502
```

4 Temperaturas

Solución de la cátedra, corregida

Enunciado (tal cual)

Ingresar un conjunto de temperaturas en una lista, finalizar la carga cuando se reciba un 50. Sólo aceptar temperaturas entre -20 y 49 grados.

Calcular y mostrar:

- Cantidad de días con temperatura bajo cero
- Promedio de temperaturas
- Promedio de temperaturas de los días cálidos, es decir con temp. mayor a 20
- Mostrar "Si" o "No" para indicar si hubo algún día con más de 40 grados.
- La mayor temperatura de los días que no fueron cálidos
- Cantidad de días con temperatura menor al promedio

Qué se corrigió

- Al mostrar la mayor temperatura de los días no cálidos se preguntaba `if mayor:`. Si esa temperatura era 0, Python lo toma como falso y decía “Todos los días fueron cálidos”.

validacion.py

```python
def ingresar_numero_entre(mensaje, minimo, maximo):
    valor = float(input(mensaje))
    while not minimo <= valor <= maximo:
        print(f"Debe ingresar un valor entre {minimo} y {maximo}")
        valor = float(input(mensaje))
    return valor
```

principal.py

```python
from validacion import ingresar_numero_entre

FIN = 50


def cargar_temperaturas():
    temperaturas = []
    print("Temperaturas entre -20 y 49 (50 para terminar)")
    t = ingresar_numero_entre("Temperatura: ", -20, FIN)
    while t != FIN:
        temperaturas.append(t)
        t = ingresar_numero_entre("Temperatura: ", -20, FIN)
    return temperaturas


def promedio(lista):
    return sum(lista) / len(lista) if lista else 0


def main():
    temps = cargar_temperaturas()
    if not temps:
        print("No se cargaron temperaturas")
        return
    prom = promedio(temps)
    calidos = [t for t in temps if t > 20]
    no_calidos = [t for t in temps if t <= 20]

    print("Días bajo cero:", len([t for t in temps if t < 0]))
    print(f"Promedio: {prom:.2f}")
    print(f"Promedio de los días cálidos: {promedio(calidos):.2f}" if calidos else "No hubo días
cálidos")
    print("¿Hubo días de más de 40 grados?", "Sí" if any(t > 40 for t in temps) else "No")
    # "is not None"/lista vacía: si la mayor no cálida es 0 grados también hay que mostrarla
    print(f"Mayor de los días no cálidos: {max(no_calidos)}" if no_calidos else "Todos los días
fueron cálidos")
    print("Días por debajo del promedio:", len([t for t in temps if t < prom]))


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Temperaturas entre -20 y 49 (50 para terminar)
Temperatura: 25
Temperatura: -5
Temperatura: 30
Temperatura: 42
Temperatura: 0
Temperatura: -2
Temperatura: 18
Temperatura: 60
Debe ingresar un valor entre -20 y 50
Temperatura: 15
Temperatura: 50
Días bajo cero: 2
Promedio: 15.38
Promedio de los días cálidos: 32.33
¿Hubo días de más de 40 grados? Sí
Mayor de los días no cálidos: 18.0
Días por debajo del promedio: 4
```

Archivos

5 Personas (versión 1, sin estructuras de datos)

Solución de la cátedra, reescrita

Enunciado (tal cual)

El archivo personas.csv contiene un padrón de personas a razón de una persona por línea y en cada una separadas con comas el documento, nombre, apellido y edad.

Desarrollar un programa en python que solicite un apellido y muestre por pantalla el nombre y edad de todas las personas cuyo apellido sea igual al ingresado.

El programa debe funcionar sin almacenar los datos de las personas en ninguna estructura de datos.

Para tener en cuenta

- El archivo está en mayúsculas: el apellido ingresado se pasa a mayúscula antes de comparar (si no, "gil" no encuentra a nadie).
- Se usa `with open`, que cierra el archivo solo.

principal.py

```
def main():
    apellido = input("Apellido a buscar: ").upper()        # el archivo está en mayúsculas
    encontrados = 0
    with open("personas.csv", encoding="utf-8") as archivo:
        for linea in archivo:                               # se procesa línea por línea, sin
guardar nada
            documento, nombre, ape, edad = linea.strip().split(",")
            if ape == apellido:
                print(f"   {nombre}: {edad} años")
                encontrados += 1
    print(f"{encontrados} persona(s) con apellido {apellido}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Apellido a buscar: gil
   RUBEN: 14 años
   AMPARO: 51 años
   ELENA: 69 años
   MONICA: 95 años
   ALBERTO: 58 años
   HUGO: 25 años
   NOELIA: 61 años
   LORENA: 41 años
   ANDREA: 94 años
   GONZALO: 11 años
   GLORIA: 32 años
   JOSE CARLOS: 32 años
   LAURA: 81 años
   ALICIA: 90 años
   ROSA: 57 años
   FRANCISCO JAVIER: 95 años
   JAVIER: 82 años
   NATALIA: 36 años
   ENCARNACION: 77 años
   FRANCISCO JOSE: 18 años
   MARIA ROSA: 52 años
   CLAUDIA: 31 años
   RAUL: 73 años
   LORENZO: 68 años
24 persona(s) con apellido GIL
```

6 Archivo de números

Solución de la cátedra, reescrita

Enunciado (tal cual)

Desarrollar un programa que lea todo el contenido del archivo numeros.txt y almacene todos los números en una lista.

A continuación, desde la lista calcular e imprimir:

- Promedio de todos los números
- Cantidad de números mayores al promedio
- Generar y mostrar una nueva lista que contenga todos los números pares

principal.py

```
def leer_numeros(nombre_archivo):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return [int(linea) for linea in archivo if linea.strip()]


def main():
    numeros = leer_numeros("numeros.txt")
    promedio = sum(numeros) / len(numeros)
    pares = [n for n in numeros if n % 2 == 0]
    print("Números leídos:", len(numeros))
    print(f"Promedio: {promedio:.2f}")
    print("Mayores al promedio:", len([n for n in numeros if n > promedio]))
    print(f"Pares ({len(pares)}):", pares)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Números leídos: 1000
Promedio: 503.50
Mayores al promedio: 503
Pares (495): [216, 978, 744, 536, 280, 618, 270, 68, 658, 218, 454, 168, 626, 280, 440, 54, 316,
868, 368, 780, 448, 496, 570, 194, 140, 522, 632, 694, 980, 792, 236, 546, 788, 624, 270, 748,
166, 602, 574, 646, 290, 326, 762, 994, 692, 666, 170, 540, 254, 308, 796, 698, 520, 694, 500,
594, 386, 792, 14, 538, 492, 948, 222, 956, 624, 170, 254, 416, 456, 868, 904, 984, 628, 296,
938, 404, 824, 268, 356, 370, 322, 546, …
```

7 Archivo de códigos postales (CSV)

Solución de la cátedra, reescrita

Enunciado (tal cual)

El archivo cp.csv contiene el listado de los códigos postales de tres provincias argentinas. En cada línea se encuentran, separados por un símbolo de punto y coma (;) los siguientes datos:

- Provincia, representada por una letra mayúscula según el estándar ISO correspondiente.
- Código, es un número entero de cuatro dígitos que identifica una localidad o varias localidades vecinas. Puede estar repetido.
- Nombre, es el nombre de la localidad correspondiente a ese código.

Se requiere leer todo el contenido del archivo y guardarlo en una lista que contenga un elemento por cada línea. En cada elemento debe almacenarse alguna estructura de datos que permita acceder individualmente a cada dato que conforma un codigo postal.

Luego de la carga el programa debe permitir el ingreso de uno o más códigos numéricos y listar provincia y nombre de todas las localidades asignadas a dichos códigos.

principal.py

```python
def leer_codigos(nombre_archivo):
    """Una tupla (provincia, código, localidad) por línea."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return [tuple(linea.strip().split(";")) for linea in archivo if linea.strip()]


def buscar(codigos, buscado):
    return [c for c in codigos if c[1] == buscado]


def main():
    codigos = leer_codigos("cp.csv")
    buscado = input("Código a buscar (vacío para terminar): ")
    while buscado:
        encontrados = buscar(codigos, buscado)
        for provincia, codigo, localidad in encontrados:
            print(f"   {provincia}: {localidad}")
        if not encontrados:
            print("   No se encontró ninguna localidad")
        buscado = input("Código a buscar (vacío para terminar): ")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Código a buscar (vacío para terminar): 9100
   No se encontró ninguna localidad
Código a buscar (vacío para terminar): 4700
   K: BANDA VARELA
   K: BARRIO LA VIÑITA
   K: CATAMARCA
   K: CHACABUCO
   K: EL TALA (SAN FERNANDO DEL VALLE DE CATAMARCA, DPTO. CAPITAL)
   K: LA AGUADA (SAN FERNANDO DEL VALLE DE CATAMARCA-DPTO.CAPITAL)
   K: LA BREA (SAN FERNANDO DEL VALLE DE CATAMARCA-DPTO.CAPITAL)
   K: LA CALERA (SAN FERNANDO DEL VALLE DE CATAMARCA-DP.CAPITAL)
   K: LA CHACARITA
   K: LA CHACARITA DE LOS PADRES
   K: LAS TEJAS DEL VALLE VIEJO
   K: LAS VARITAS
   K: LAZARETO
   K: LOMA CORTADA
   K: RIO DEL TALA
   K: SAN FERNANDO DEL VALLE DE CATAMARCA
   K: VILLA CUBAS
   K: VILLA PARQUE CHACABUCO
Código a buscar (vacío para terminar): 1234
   No se encontró ninguna localidad
Código a buscar (vacío para terminar):
```

8 Análisis de propinas

Solución de la cátedra, reescrita

Enunciado (tal cual)

El archivo tips.csv contiene los datos de las propinas recibidas por los empleados de una pizzería en New York.

El archivo contiene los siguientes campos, separados por comas

- "total_bill" → Total del servicio,
- "tip" → Propina,
- "sex" → Sexo del cliente,
- "smoker" → El cliente es fumador,
- "day" → Día (Juev, Vie, Sab, Dom),
- "time" → Horario,
- "size" → Tamaño de la mesa (comensales)

Debe desarrollar un programa que desde los datos del archivo informe:

- ¿Quién paga más propinas?¿Hombres o mujeres?
- ¿Cuáles son los días más 'lentos', con menos propinas?
- ¿Cuál es el promedio de las propinas?
- ¿Cuándo se vendió la orden más grande (qué día y en qué turno)?

Para tener en cuenta

- La versión de la cátedra usa la librería `rich` (hay que instalarla con `pip install rich`); acá se usa `print` y el módulo `csv`.

principal.py

```
import csv

DIAS = {"Juev": "jueves", "Vie": "viernes", "Sab": "sábado", "Dom": "domingo"}


def main():
    por_sexo = {"Hombre": 0.0, "Mujer": 0.0}
    por_dia = {dia: 0.0 for dia in DIAS}
    total_propinas, cantidad = 0.0, 0
    mayor = None                                    # la fila con la orden más grande

    with open("tips.csv", encoding="utf-8") as archivo:
        for fila in csv.DictReader(archivo):        # la primera línea tiene los títulos
            propina = float(fila["tip"])
            por_sexo[fila["sex"]] += propina
            por_dia[fila["day"]] += propina
            total_propinas += propina
            cantidad += 1
            if mayor is None or float(fila["total_bill"]) > float(mayor["total_bill"]):
                mayor = fila

    print(f"Propinas de hombres: {por_sexo['Hombre']:.2f} · de mujeres: {por_sexo['Mujer']:.2f}")
    print("Pagan más propinas:", "los hombres" if por_sexo["Hombre"] > por_sexo["Mujer"] else
"las mujeres")
    lento = min(por_dia, key=por_dia.get)
    print(f"Día más lento: {DIAS[lento]} ({por_dia[lento]:.2f} de propinas)")
    print(f"Propina promedio: {total_propinas / cantidad:.2f}")
    print(f"Orden más grande: {mayor['total_bill']} el {DIAS[mayor['day']]}, turno
{mayor['time'].lower()}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Propinas de hombres: 485.07 · de mujeres: 246.51
Pagan más propinas: los hombres
Día más lento: viernes (51.96 de propinas)
Propina promedio: 3.00
Orden más grande: 50.81 el sábado, turno cena
```

9 Archivo de códigos postales (JSON)

Solución de la cátedra, reescrita

Enunciado (tal cual)

El archivo cp.json contiene el listado de los códigos postales de tres provincias argentinas en formato JSON con una lista de códigos, cada uno de los cuales posee:

- Provincia, representada por una letra mayúscula según el estándar ISO correspondiente.
- Código, es un número entero de cuatro dígitos que identifica una localidad o varias localidades vecinas. Puede estar repetido.
- Nombre, es el nombre de la localidad correspondiente a ese código.

Se requiere leer todo el contenido del archivo y guardarlo en una lista que contenga un elemento por cada línea. En cada elemento debe almacenarse alguna estructura de datos que permita acceder individualmente a cada dato que conforma un codigo postal.

Luego de la carga el programa debe permitir el ingreso de uno o más códigos numéricos y listar provincia y nombre de todas las localidades asignadas a dichos códigos.

Para tener en cuenta

- En el JSON el código postal es un número: el texto ingresado se convierte con `int()` antes de comparar.

principal.py

```python
import json


def leer_codigos(nombre_archivo):
    """El JSON es una lista de diccionarios: json.load la devuelve lista para usar."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return json.load(archivo)


def main():
    codigos = leer_codigos("cp.json")
    buscado = input("Código a buscar (vacío para terminar): ")
    while buscado:
        # en el JSON el código es un número: hay que comparar con int, no con el texto
        encontrados = [c for c in codigos if c["codigo_postal"] == int(buscado)]
        for c in encontrados:
            print(f"   {c['codigo_provincia']}: {c['nombre_localidad']}")
        if not encontrados:
            print("   No se encontró ninguna localidad")
        buscado = input("Código a buscar (vacío para terminar): ")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Código a buscar (vacío para terminar): 4743
   K: ACONQUIJA
   K: ALTO DE LA JUNTA
   K: ALUMBRERA
   K: BUENA VISTA (EL ALAMITO, DPTO. ANDALGALA)
   K: EL ALAMITO
   K: EL ARBOLITO (EL ALAMITO, DPTO. ANDALGALA)
   K: EL ESPINILLO
   K: EL SUNCHO (ALTO DE LA JUNTA, DPTO. ANDALGALA)
   K: LAS ESTANCIAS
   K: LAS PAMPITAS (EL ALAMITO-DPTO.ANDALGALA)
   K: RIO POTRERO
Código a buscar (vacío para terminar): 1234
   No se encontró ninguna localidad
Código a buscar (vacío para terminar):
```

10 JSON multinivel

Solución de la cátedra, reescrita

Enunciado (tal cual)

Del archivo resultset.json informar:

- Todo el contenido del json
- El contenido de la rama "ResultSet"
- El atributo FileSize del segundo item de la lista de "Result"
- El valor del atributo "Url" del "Thumbnail" del item del punto anterior

principal.py

```
import json


def main():
    with open("resultset.json", encoding="utf-8") as archivo:
        datos = json.load(archivo)                    # diccionarios y listas anidados

    print("Todo el contenido:")
    print(json.dumps(datos, indent=2, ensure_ascii=False))
    print("\nLa rama ResultSet tiene las claves:", list(datos["ResultSet"].keys()))
    segundo = datos["ResultSet"]["Result"][1]         # las listas empiezan en 0
    print("FileSize del segundo resultado:", segundo["FileSize"])
    print("Url del Thumbnail de ese resultado:", segundo["Thumbnail"]["Url"])


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Todo el contenido:
{
  "ResultSet": {
    "totalResultsAvailable": "1827221",
    "totalResultsReturned": 2,
    "firstResultPosition": 1,
    "Result": [
      {
        "Title": "potato jpg",
        "Summary": "Kentang Si bungsu dari keluarga Solanum tuberosum L ini ternyata memiliki
khasiat untuk mengurangi kerutan  jerawat  bintik hitam dan kemerahan pada kulit  Gunakan
seminggu sekali sebagai",
        "Url": "http://www.aprenderaprogramar.com/spaw/uploads /images/potato.jpg",
        "ClickUrl": "http://www. aprenderaprogramar.com/spaw/uploads/images/potato.jpg",
        "RefererUrl": "http://www.mediaindonesia.com/mediaperempuan/index.php?ar_id=Nzkw",
        "FileSize": 22630,
        "FileFormat": "jpeg",
        "Height": "362",
        "Width": "532",
        "Thumbnail": {
          "Url": "http://thm-a01.yimg.com/nimage/557094559c18f16a",
          "Height": "98",
          "Width": "145"
        }
      },
      {
        "Title": "potato jpg",
        "Summary": "Introduction of puneri aloo This is a traditional potato preparation
flavoured with curry leaves and peanuts and can be eaten on fasting day  Preparation time   10
min",
… (14 renglones más)
  }
}

La rama ResultSet tiene las claves: ['totalResultsAvailable', 'totalResultsReturned',
'firstResultPosition', 'Result']
FileSize del segundo resultado: 119398
Url del Thumbnail de ese resultado: http://thm-a01.yimg.com/nimage/7fa23212efe84b64
```

11 Palabras del Quijote (archivos y conjuntos)

Solución de la cátedra, corregida

Enunciado (tal cual)

Tomando el texto del libro "El Quijote de La Mancha" y el diccionario del idioma inglés calcular e informar

- Cantidad de palabras únicas (sin repetición) del libro.
- Cantidad de palabras del diccionario.
- Cantidad de palabras del libro que no existen en el diccionario.
- Listado ordenados de todas las palabras que no existen.

Qué se corrigió

- Es el mismo ejercicio en las carpetas *archivos* y *conjuntos*: está una sola vez.
- En la versión de conjuntos se separaba con `split()` (“Sancho,” y “sancho” eran palabras distintas) y se imprimía el conjunto entero. En la de archivos no se pasaba a minúscula. Acá se pasa a minúscula y se toman solo las letras (`re.findall`).
- El `quijote.txt` es la traducción al inglés: por eso se compara con un diccionario de inglés.

principal.py

```python
import re


def palabras_de(nombre_archivo):
    """Todas las palabras del texto, en minúscula y sin signos de puntuación."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return re.findall(r"[a-záéíóúüñ]+", archivo.read().lower())


def leer_diccionario(nombre_archivo):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return {linea.strip() for linea in archivo if linea.strip()}


def main():
    libro = set(palabras_de("quijote.txt"))          # el texto es la traducción al inglés
    diccionario = leer_diccionario("words_alpha.txt")
    no_existen = libro - diccionario                 # diferencia de conjuntos

    print("Palabras únicas del libro:", len(libro))
    print("Palabras del diccionario:", len(diccionario))
    print("Palabras del libro que no están en el diccionario:", len(no_existen))
    print("Listado ordenado:")
    for palabra in sorted(no_existen):
        print("  ", palabra)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Palabras únicas del libro: 15715
Palabras del diccionario: 370105
Palabras del libro que no están en el diccionario: 1210
Listado ordenado:
   abajo
   abernuncio
   abideth
   abindarraez
   abrenuncio
   achilleses
   acquaviva
   adriani
   adversae
   aequo
   agilers
   agrajes
   agramante
   aguero
   aguilar
   ainting
   alagones
   alastrajareas
   albogue
   albogues
   albraca
   albracca
… (1182 renglones más)
   zonzorino
   zopyrus
   zoraida
   zoroastric
   zorruna
   zulema
```

Conjuntos

12 Eliminación de repetidos

Solución de la cátedra

Enunciado (tal cual)

Generar 10 números enteros al azar entre 1 y 20. Guardarlos en un conjunto y mostrar tanto la cantidad de elementos originales como los del conjunto para verificar la eliminación de repetidos.

principal.py

```
import random

random.seed(7)       # para que la salida sea siempre la misma (sacarlo para que cambie)

numeros = [random.randint(1, 20) for _ in range(10)]
conjunto = set(numeros)              # el conjunto no guarda repetidos

print("Lista original:", numeros)
print("Conjunto:", conjunto)
print("Elementos originales:", len(numeros), "· sin repetidos:", len(conjunto))
```

Salida al correrlo

```
Lista original: [11, 5, 13, 2, 3, 18, 4, 12, 19, 2]
Conjunto: {2, 3, 4, 5, 11, 12, 13, 18, 19}
Elementos originales: 10 · sin repetidos: 9
```

13 Números con un conjunto

Solución de la cátedra, corregida

Enunciado (tal cual)

Del archivo numeros.txt que contiene un número entero por cada línea, generar un conjunto que contenga todos los números del archivo y desde el conjunto calcular e informar:

- Cantidad de números no repetidos
- Suma de todos los números
- Cantidad de impares
- Promedio de pares

Qué se corrigió

- La suma, los impares y el promedio de pares se calculaban con todos los números leídos, repetidos incluidos, y no desde el conjunto como pide el enunciado.

principal.py

```python
def leer_numeros(nombre_archivo):
    """Un número entero por línea -> conjunto (los repetidos quedan una sola vez)."""
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return {int(linea) for linea in archivo if linea.strip()}


def main():
    numeros = leer_numeros("numeros.txt")
    pares = [n for n in numeros if n % 2 == 0]
    print("Cantidad de números no repetidos:", len(numeros))
    print("Suma de todos los números:", sum(numeros))
    print("Cantidad de impares:", len([n for n in numeros if n % 2 != 0]))
    print(f"Promedio de los pares: {sum(pares) / len(pares):.2f}" if pares else "No hay pares")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Cantidad de números no repetidos: 622
Suma de todos los números: 314220
Cantidad de impares: 318
Promedio de los pares: 507.45
```

14 Operaciones de conjuntos

Solución propuesta (la cátedra no la publicó)

Enunciado (tal cual)

Palabras Únicas: Toma una cadena como entrada y crea un conjunto que contenga las palabras únicas presentes en esa cadena.

Palabras Comunes: Dados dos textos, encuentra las palabras que son comunes en ambos textos utilizando conjuntos.

Elementos Únicos en Listas: Toma dos listas y crea un conjunto que contenga los elementos que son únicos en ambas listas (no presentes en la otra lista).

Para tener en cuenta

- Es del práctico de clase (no está en el repositorio).
- Un conjunto no tiene orden: al imprimirlo, los elementos pueden salir en otro orden.

principal.py

```python
def palabras_unicas(cadena):
    """Conjunto con las palabras de la cadena, sin repetir."""
    return set(cadena.lower().split())


def palabras_comunes(texto1, texto2):
    """Palabras que aparecen en los dos textos (intersección)."""
    return palabras_unicas(texto1) & palabras_unicas(texto2)


def elementos_unicos(lista1, lista2):
    """Elementos que están en una sola de las dos listas (diferencia simétrica)."""
    return set(lista1) ^ set(lista2)


def main():
    print(palabras_unicas("el perro y el gato y el loro"))
    print(palabras_comunes("me gusta programar en Python", "en Python todo es un objeto"))
    print(elementos_unicos([1, 2, 3, 4, 5], [4, 5, 6, 7]))


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
{'el', 'perro', 'loro', 'gato', 'y'}
{'python', 'en'}
{1, 2, 3, 6, 7}
```

Diccionarios

15 Agenda de países

Solución de la cátedra

Enunciado (tal cual)

Crear un diccionario hardcodeado con los 10 países más grandes de Sudamérica, indexado por su código ISO. Permitir al usuario consultar por código en un ciclo `while` hasta que se ingrese una cadena vacía.

principal.py

```
paises = {
    "BR": "Brasil",
    "AR": "Argentina",
    "CO": "Colombia",
    "PE": "Perú",
    "VE": "Venezuela",
    "CL": "Chile",
    "EC": "Ecuador",
    "BO": "Bolivia",
    "PY": "Paraguay",
    "UY": "Uruguay",
}

codigo = input("Código del país (vacío para salir): ")
while codigo:
    nombre = paises.get(codigo.upper())
    if nombre:
        print(nombre)
    else:
        print("País no encontrado")
    codigo = input("Código del país (vacío para salir): ")
```

Salida al correrlo

```
Código del país (vacío para salir): ar
Argentina
Código del país (vacío para salir): BR
Brasil
Código del país (vacío para salir): MX
País no encontrado
Código del país (vacío para salir):
```

16 Calculadora con diccionario de operaciones

Solución de la cátedra

Enunciado (tal cual)

Programar una calculadora que almacene las operaciones aritméticas en un diccionario indexado por el símbolo del operador. El usuario ingresa una operación indicando primer operando, operador y segundo operando separados por espacios (usar `split`). La carga finaliza con una operación vacía. En caso de división por cero, retornar `float('inf')`.

principal.py

```
def suma(a, b):
    return a + b

def resta(a, b):
    return a - b

def multiplicacion(a, b):
    return a * b

def division(a, b):
    return float('inf') if b == 0 else a / b

operaciones = {
    "+": suma,
    "-": resta,
    "*": multiplicacion,
    "/": division,
}

while True:
    linea = input("Operación (vacío para salir): ")
    if not linea:
        break

    partes = linea.split()
    if len(partes) != 3:
        print("Formato inválido. Use: operando operador operando")
        continue

    operando1 = float(partes[0])
    operador  = partes[1]
    operando2 = float(partes[2])

    funcion = operaciones.get(operador)
    if funcion:
        print("Resultado:", funcion(operando1, operando2))
    else:
        print("Operador no válido")
```

Salida al correrlo

```
Operación (vacío para salir): 3 + 4
Resultado: 7.0
Operación (vacío para salir): 10 / 0
Resultado: inf
Operación (vacío para salir): 2 ^ 3
Operador no válido
Operación (vacío para salir): 5 *
Formato inválido. Use: operando operador operando
Operación (vacío para salir): 7.5 * 2
Resultado: 15.0
Operación (vacío para salir):
```

17 Padrón de personas (versión 2, con diccionario)

Solución de la cátedra, reescrita

Enunciado (tal cual)

El archivo personas.csv contiene un padrón de personas a razón de una persona por línea y en cada una separadas con comas el documento, nombre, apellido y edad. Desarrollar un programa en python que lea el archivo y guarde todo su contenido en un diccionario indexado por documento. Luego el programa debe ofrecer un menú con las siguientes opciones:

- Búsqueda por documento: que solicite un documento y si lo encuentra muestre todos los datos de la persona encontrada y un mensaje adecuado si no la encuentra.
- Búsqueda por apellido: que solicite un apellido y muestre por pantalla todos los datos de todas las personas cuyo apellido sea igual al ingresado.
- Mostrar el promedio de edades de todos.

Para tener en cuenta

- La búsqueda por apellido pasa el texto a mayúscula (el archivo está en mayúsculas).

principal.py

```
def leer_padron(nombre_archivo):
    """documento,nombre,apellido,edad -> diccionario indexado por documento."""
    padron = {}
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for linea in archivo:
            if linea.strip():
                documento, nombre, apellido, edad = linea.strip().split(",")
                padron[documento] = {"nombre": nombre, "apellido": apellido, "edad": int(edad)}
    return padron


def mostrar(documento, persona):
    print(f"   {documento}: {persona['apellido']}, {persona['nombre']} ({persona['edad']} años)")


def main():
    padron = leer_padron("personas.csv")
    opcion = ""
    while opcion != "0":
        print("1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir")
        opcion = input("Opción: ")
        if opcion == "1":
            documento = input("Documento: ")
            if documento in padron:                        # búsqueda directa por clave
                mostrar(documento, padron[documento])
            else:
                print("   No hay ninguna persona con ese documento")
        elif opcion == "2":
            apellido = input("Apellido: ").upper()
            encontrados = {d: p for d, p in padron.items() if p["apellido"] == apellido}
            for documento, persona in encontrados.items():
                mostrar(documento, persona)
            print(f"   {len(encontrados)} persona(s) con apellido {apellido}")
        elif opcion == "3":
            promedio = sum(p["edad"] for p in padron.values()) / len(padron)
            print(f"   Promedio de edades: {promedio:.2f}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 1
Documento: 22861213
   22861213: GIL, RUBEN (14 años)
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 2
Apellido: rubio
   35566056: RUBIO, MANUELA (52 años)
   68351740: RUBIO, EVA MARIA (22 años)
   19353929: RUBIO, LAURA (100 años)
   91950530: RUBIO, ANTONIA (19 años)
   53490637: RUBIO, MARINA (72 años)
   61079344: RUBIO, ANTONIA (65 años)
   34853102: RUBIO, LAURA (35 años)
   71078117: RUBIO, MARIA NIEVES (36 años)
   23898222: RUBIO, GLORIA (10 años)
   31605936: RUBIO, GREGORIO (78 años)
   54830247: RUBIO, ANA ISABEL (87 años)
   25892921: RUBIO, EVA (28 años)
   69070710: RUBIO, GREGORIO (51 años)
   34188093: RUBIO, CAROLINA (92 años)
   82489974: RUBIO, ANGELA (78 años)
   75749197: RUBIO, OSCAR (91 años)
   71101677: RUBIO, JORDI (93 años)
   59048581: RUBIO, RAFAEL (24 años)
   44529796: RUBIO, MARIA ANGELES (69 años)
   18090156: RUBIO, EVA (29 años)
   94655512: RUBIO, CRISTINA (75 años)
   90752055: RUBIO, ENRIQUE (61 años)
   94003566: RUBIO, MARIA JESUS (66 años)
   13005375: RUBIO, MARINA (50 años)
   43663990: RUBIO, CELIA (76 años)
   73437027: RUBIO, MARIA ISABEL (28 años)
   26 persona(s) con apellido RUBIO
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 3
   Promedio de edades: 54.88
1. Buscar por documento  2. Buscar por apellido  3. Promedio de edades  0. Salir
Opción: 0
```

18 Conteo de códigos postales por provincia

Solución de la cátedra

Enunciado (tal cual)

Del archivo de codigos postales contar los códigos de cada provincia usando un diccionario de contadores.

principal.py

```
NOMBRES = {"X": "Córdoba", "H": "Chubut", "K": "Catamarca"}


def main():
    cantidad = {}                                   # diccionario de contadores: provincia ->
cantidad
    with open("cp.csv", encoding="utf-8") as archivo:
        for linea in archivo:
            if linea.strip():
                provincia = linea.split(";")[0]
                cantidad[provincia] = cantidad.get(provincia, 0) + 1    # .get: sirve aunque la
provincia sea nueva
    for provincia, cant in sorted(cantidad.items()):
        print(f"Códigos de {NOMBRES.get(provincia, provincia)}: {cant}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Códigos de Chubut: 448
Códigos de Catamarca: 783
Códigos de Córdoba: 2357
```

Programación funcional

19 Calculadora

Solución de la cátedra, corregida

Enunciado (tal cual)

Desarollar un programa que implemente una calculadora básica. El mismo debe presentar un menú de opciones con una opción por cada operación aritmética básica (suma, diferencia, producto y cociente).

Qué se corrigió

- Una opción fuera del menú (por ejemplo 9) hacía que el programa se cortara con un error.

principal.py

```
# Cada opción del menú es una función guardada en un diccionario
operaciones = {
    "1": ("Sumar", lambda a, b: a + b),
    "2": ("Restar", lambda a, b: a - b),
    "3": ("Multiplicar", lambda a, b: a * b),
    "4": ("Dividir", lambda a, b: a / b if b != 0 else float("inf")),
}


def main():
    print("Calculadora básica")
    for clave, (nombre, _) in operaciones.items():
        print(f"{clave}- {nombre}")
    print("0- Salir")

    opcion = input("Opción: ")
    while opcion != "0":
        if opcion in operaciones:
            a = float(input("Primer operando: "))
            b = float(input("Segundo operando: "))
            nombre, funcion = operaciones[opcion]
            print(f"{nombre}: {funcion(a, b)}")
        else:
            print("Opción no válida")                # la de la cátedra se rompía con una opción
fuera de rango
        opcion = input("Opción: ")
    print("Gracias")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Calculadora básica
1- Sumar
2- Restar
3- Multiplicar
4- Dividir
0- Salir
Opción: 1
Primer operando: 3
Segundo operando: 4
Sumar: 7.0
Opción: 4
Primer operando: 10
Segundo operando: 0
Dividir: inf
Opción: 7
Opción no válida
Opción: 0
Gracias
```

20 Tito el robot

Solución de la cátedra, corregida

Enunciado (tal cual)

Desarrollar un programa controlado por menú de opciones que permita simular el desplazamiento de un robot sobre un plano cuyo tamaño es de 400 × 400, y las posiciones válidas se identifican desde 0 como en un sistema de coordenadas cartesianas. Inicialmente se genera la posición aleatoria del robot en forma de punto (x,y). Luego se presenta un menú de opciones que permita los siguientes movimientos:

- Opción N: Girar al norte y avanzar 10 pasos
- Opción S: Girar al sur y avanzar 20 pasos
- Opción E: Girar al este y avanzar 10 pasos
- Opción O: Girar al oeste y avanzar 20 pasos
- Opcion F: Finalizar

Luego de cada movimiento el programa debe informar la posición del robot.

Alternativas:

- En lugar de ingresar cada movimiento individualmente, se puede recibir una cadena que contenga más de un movimiento, por ejemplo "NNEENSSNO".
- En el caso de que una orden de movimiento lleve al robot fuera de los márgenes del plano, el movimiento debería finalizar en el borde.

Qué se corrigió

- La opción O (oeste) **sumaba** 20 a x en lugar de restarlos.
- Se agregan las dos alternativas: varios movimientos en una cadena y frenar en el borde del plano.

principal.py

```
import random

random.seed(3)       # para que la salida sea siempre la misma (sacarlo para que cambie)
LADO = 400           # el plano va de 0 a 400 en x y en y

# Cada movimiento es una función que recibe la posición y devuelve la nueva
movimientos = {
    "N": lambda x, y: (x, y + 10),
    "S": lambda x, y: (x, y - 20),
    "E": lambda x, y: (x + 10, y),
    "O": lambda x, y: (x - 20, y),        # al oeste x disminuye (en la de la cátedra sumaba)
}


def dentro_del_plano(x, y):
    """Si un movimiento lo saca del plano, queda en el borde."""
    return min(max(x, 0), LADO), min(max(y, 0), LADO)


def main():
    x, y = random.randint(0, LADO), random.randint(0, LADO)
    print(f"Posición inicial: ({x}, {y})")
    print("N- Norte (10) · S- Sur (20) · E- Este (10) · O- Oeste (20) · F- Finalizar")
    orden = input("Movimientos: ").upper()
    while orden != "F":
        for letra in orden:                        # acepta varios movimientos juntos, ej.
"NNEO"
            if letra in movimientos:
                x, y = dentro_del_plano(*movimientos[letra](x, y))
                print(f"   {letra} -> ({x}, {y})")
        orden = input("Movimientos: ").upper()
    print("Gracias")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Posición inicial: (121, 303)
N- Norte (10) · S- Sur (20) · E- Este (10) · O- Oeste (20) · F- Finalizar
Movimientos: N
   N -> (121, 313)
Movimientos: OOO
   O -> (101, 313)
   O -> (81, 313)
   O -> (61, 313)
Movimientos: eesn
   E -> (71, 313)
   E -> (81, 313)
   S -> (81, 293)
   N -> (81, 303)
Movimientos: SSSSSSSSSSSSSSSSSSSSSS
   S -> (81, 283)
   S -> (81, 263)
   S -> (81, 243)
   S -> (81, 223)
   S -> (81, 203)
   S -> (81, 183)
   S -> (81, 163)
   S -> (81, 143)
   S -> (81, 123)
   S -> (81, 103)
   S -> (81, 83)
   S -> (81, 63)
   S -> (81, 43)
   S -> (81, 23)
   S -> (81, 3)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
   S -> (81, 0)
Movimientos: F
Gracias
```

21 Lista de números

Solución de la cátedra, corregida

Enunciado (tal cual)

Generar 1000 números enteros al azar entre -1000000 y 1000000. Investigar cómo generar los números con un flujo en lugar de un ciclo for Informar:

- El menor de todos.
- La cantidad de pares.
- El promedio de los impares.
- El cuadrado de todos los que se encuentren entre 10 y 100 sin incluirlos
- Los múltiplos de 3 del punto anterior (los multiplos de 3 de los cuadrados calculados)
- Todos los múltiplos de 7 ordenados en forma descendente.
- El promedio de los impares negativos.
- La desviación estandar de todos.
- Si existe o no algún múltiplo de 127.
- Generar una lista que contenga únicamente los que terminan en 2 o 3.

Qué se corrigió

- Buscaba múltiplos de 14543527 en lugar de 127.
- Con números entre -1.000.000 y 1.000.000 casi nunca sale uno entre 10 y 100: por eso esa lista (y la de sus múltiplos de 3) da vacía.

principal.py

```python
import math
import random
from functools import reduce

random.seed(2026)        # para que la salida sea siempre la misma (sacarlo para que cambie)

es_par = lambda x: x % 2 == 0
promedio = lambda lista: sum(lista) / len(lista) if lista else 0

numeros = [random.randint(-1_000_000, 1_000_000) for _ in range(1000)]   # comprensión en vez de un for

print("Menor de todos:", reduce(lambda a, b: a if a < b else b, numeros))
print("Cantidad de pares:", len(list(filter(es_par, numeros))))
impares = list(filter(lambda x: not es_par(x), numeros))
print(f"Promedio de los impares: {promedio(impares):.2f}")
cuadrados = list(map(lambda x: x ** 2, filter(lambda x: 10 < x < 100, numeros)))
print("Cuadrados de los que están entre 10 y 100:", cuadrados)
print("Múltiplos de 3 de esos cuadrados:", list(filter(lambda x: x % 3 == 0, cuadrados)))
multiplos_7 = sorted(filter(lambda x: x % 7 == 0, numeros), reverse=True)
print(f"Múltiplos de 7 (de mayor a menor), {len(multiplos_7)}:", multiplos_7[:10], "…")
print(f"Promedio de los impares negativos: {promedio(list(filter(lambda x: x < 0, impares))):.2f}")
media = promedio(numeros)
desvio = math.sqrt(sum(map(lambda x: (x - media) ** 2, numeros)) / (len(numeros) - 1))
print(f"Desviación estándar: {desvio:.2f}")
print("¿Hay algún múltiplo de 127?", "Sí" if any(map(lambda x: x % 127 == 0, numeros)) else "No")
terminan_2_o_3 = list(filter(lambda x: abs(x) % 10 in (2, 3), numeros))
print(f"Terminan en 2 o 3: {len(terminan_2_o_3)} números, por ejemplo", terminan_2_o_3[:8])
```

Salida al correrlo

```
Menor de todos: -999552
Cantidad de pares: 480
Promedio de los impares: -1042.27
Cuadrados de los que están entre 10 y 100: []
Múltiplos de 3 de esos cuadrados: []
Múltiplos de 7 (de mayor a menor), 139: [999999, 992159, 980189, 954919, 954163, 952357, 949228, 943971, 916706, 854826] …
Promedio de los impares negativos: -488528.61
Desviación estándar: 566323.73
¿Hay algún múltiplo de 127? Sí
Terminan en 2 o 3: 209 números, por ejemplo [-329993, 357272, 998482, 854133, 260452, 642853, 200763, 148843]
```

22 Personas (versión 3, funcional)

Solución de la cátedra, corregida

Enunciado (tal cual)

Reescribir el ejercicio del padrón de personas aplicando programación funcional, tanto para el menú como para los procesos.

Qué se corrigió

- Al elegir Salir mostraba también “opción no válida”.

principal.py

```
import csv


def cargar(nombre_archivo):
    with open(nombre_archivo, encoding="utf-8") as archivo:
        return {d: {"nombre": n, "apellido": a, "edad": int(e)} for d, n, a, e in
csv.reader(archivo)}


def mostrar(documento, p):
    print(f"   {documento}: {p['apellido']}, {p['nombre']} ({p['edad']} años)")


def por_documento(datos):
    documento = input("Documento: ")
    persona = datos.get(documento)
    mostrar(documento, persona) if persona else print("   Persona no encontrada")


def por_apellido(datos):
    apellido = input("Apellido: ").upper()
    encontrados = list(filter(lambda item: item[1]["apellido"] == apellido, datos.items()))
    list(map(lambda item: mostrar(*item), encontrados))
    print(f"   {len(encontrados)} persona(s)")


def promedio_edades(datos):
    edades = list(map(lambda p: p["edad"], datos.values()))
    print(f"   Promedio de edades: {sum(edades) / len(edades):.2f}")


# El menú es un diccionario: opción -> (texto, función)
MENU = {"1": ("Búsqueda por documento", por_documento),
        "2": ("Búsqueda por apellido", por_apellido),
        "3": ("Promedio de edades", promedio_edades)}


def main():
    datos = cargar("personas.csv")
    opcion = ""
    while opcion != "4":
        print(" · ".join(f"{k}. {texto}" for k, (texto, _) in MENU.items()) + " · 4. Salir")
```

```
        opcion = input("Opción: ")
        if opcion in MENU:
            MENU[opcion][1](datos)
        elif opcion != "4":                    # en la de la cátedra, "Salir" mostraba "opción
no válida"
            print("   Opción no válida")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
1. Búsqueda por documento · 2. Búsqueda por apellido · 3. Promedio de edades · 4. Salir
Opción: 1
Documento: 35566056
   35566056: RUBIO, MANUELA (52 años)
1. Búsqueda por documento · 2. Búsqueda por apellido · 3. Promedio de edades · 4. Salir
Opción: 2
Apellido: rubio
   35566056: RUBIO, MANUELA (52 años)
   68351740: RUBIO, EVA MARIA (22 años)
   19353929: RUBIO, LAURA (100 años)
   91950530: RUBIO, ANTONIA (19 años)
   53490637: RUBIO, MARINA (72 años)
   61079344: RUBIO, ANTONIA (65 años)
   34853102: RUBIO, LAURA (35 años)
   71078117: RUBIO, MARIA NIEVES (36 años)
   23898222: RUBIO, GLORIA (10 años)
   31605936: RUBIO, GREGORIO (78 años)
   54830247: RUBIO, ANA ISABEL (87 años)
   25892921: RUBIO, EVA (28 años)
   69070710: RUBIO, GREGORIO (51 años)
   34188093: RUBIO, CAROLINA (92 años)
   82489974: RUBIO, ANGELA (78 años)
   75749197: RUBIO, OSCAR (91 años)
   71101677: RUBIO, JORDI (93 años)
   59048581: RUBIO, RAFAEL (24 años)
   44529796: RUBIO, MARIA ANGELES (69 años)
… (10 renglones más)
   Promedio de edades: 54.88
1. Búsqueda por documento · 2. Búsqueda por apellido · 3. Promedio de edades · 4. Salir
Opción: 9
   Opción no válida
1. Búsqueda por documento · 2. Búsqueda por apellido · 3. Promedio de edades · 4. Salir
Opción: 4
```

23 Map/filter con una lista de números

Solución propuesta (la cátedra no la publicó)

Enunciado (tal cual)

Generar una lista con todos los números naturales entre -5 y 10 ambos inclusive.

Utilizando únicamente programación funcional informar:

- Cuadrados de todos los números de la lista
- Todos los números negativos de la lista
- Sumatoria de todos los números de la lista

principal.py

```
from functools import reduce

numeros = list(range(-5, 11))            # de -5 a 10, ambos incluidos

print("Números:", numeros)
print("Cuadrados:", list(map(lambda x: x ** 2, numeros)))
print("Negativos:", list(filter(lambda x: x < 0, numeros)))
print("Sumatoria:", reduce(lambda a, b: a + b, numeros))
```

Salida al correrlo

```
Números: [-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Cuadrados: [25, 16, 9, 4, 1, 0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Negativos: [-5, -4, -3, -2, -1]
Sumatoria: 40
```

24 Map/filter/reduce con una lista de palabras

Solución propuesta (la cátedra no la publicó)

Enunciado (tal cual)

Dada la siguiente lista de palabras: "Aruba","Jamaica","Bermuda","Bahama","Key Largo","Montigo"

Utilizando únicamente programación funcional informar:

- Letra inicial de cada palabra
- Longitud de cada palabra
- Cantidad total de letras (sumando totas las palabras)

principal.py

```
from functools import reduce

palabras = ["Aruba", "Jamaica", "Bermuda", "Bahama", "Key Largo", "Montigo"]

print("Iniciales:", list(map(lambda p: p[0], palabras)))
longitudes = list(map(len, palabras))
print("Longitudes:", longitudes)
print("Total de letras:", reduce(lambda a, b: a + b, longitudes))
print("Total sin contar espacios:", reduce(lambda a, p: a + len(p.replace(" ", "")), palabras,
0))
```

Salida al correrlo

```
Iniciales: ['A', 'J', 'B', 'B', 'K', 'M']
Longitudes: [5, 7, 7, 6, 9, 7]
Total de letras: 41
Total sin contar espacios: 40
```

25 Ejemplos: funciones como argumento

Solución de la cátedra

Enunciado (tal cual)

Ejemplos de la cátedra: en Python una función es un valor, así que se puede pasar a otra función. `calc.py` aplica distintas funciones a un número y `for.py` recorre una lista aplicando la función que recibe.

calc.py

```
from math import sqrt

def cuadrado(x):
    return x ** 2

def cubo(x):
    return x ** 3

def mitad(x):
    return x / 2

## operacion es una función
def calcular(valor, operacion):
    return operacion(valor)

print(calcular(9, cuadrado))
print(calcular(9, cubo))
print(calcular(9, mitad))
print(calcular(9, lambda x: x * 2))
print(calcular(9, sqrt))
calcular(9, print)
```

Salida de calc.py

```
81
729
4.5
18
3.0
9
```

for.py

```
from math import sqrt


def recorrer_mostrar(v, funcion):
    for x in v:
        print(funcion(x))


def map(v, funcion):
    resultados = []
    for x in v:
        resultados.append(funcion(x))
    return resultados



numeros = [3,8,4,122]
print(map(numeros, lambda x: x ** 2))
```

Salida de for.py

```
[9, 64, 16, 14884]
```

26 Ejemplos: lambda y comprensión de listas

Solución de la cátedra, corregida

Enunciado (tal cual)

Los ejemplos de la clase y el ejercicio de la clase:

```
lst = ["Aruba", "Jamaica", "Bermuda", "Bahama", "Key Largo", "Montigo"]
```

1. Crear una lista por comprensión con el primer carácter de cada palabra y otra con la longitud de cada palabra; con Reduce, la cantidad total de caracteres.
2. Con random, crear por comprensión una lista de 50 números enteros al azar entre 1 y 100.
3. Con Filter, separar los mayores a 50 de los menores o iguales a 50.
4. Con Map, obtener solo los impares con el signo invertido (negativos).
5. Con Reduce, la sumatoria total de la lista del paso anterior.

Qué se corrigió

- En `comprension.py` decía “cubos” pero calculaba cuadrados.
- En `map_filter_reduce.py` los ejemplos estaban comentados y el de `map` que decía “cuadrados” calculaba si es par. Acá andan y cada uno hace lo que dice.

comprensiones.py

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Forma tradicional: un for que va armando la lista
pares = []
for n in numeros:
    if n % 2 == 0:
        pares.append(n)
print("Pares (con for):", pares)

# Por comprensión: [expresión for elemento in iterable if condición]
print("Pares (comprensión):", [n for n in numeros if n % 2 == 0])
print("Siguiente de cada par:", [n + 1 for n in numeros if n % 2 == 0])
print("Cuadrados de los múltiplos de 3:", [n ** 2 for n in numeros if n % 3 == 0])

# Comprensión de diccionarios: {clave: valor for ...}
personas = [
    {"nombre": "Alice", "edad": 30, "puesto": "Data Scientist"},
    {"nombre": "Bob", "edad": 35, "puesto": "Data Engineer"},
    {"nombre": "Chris", "edad": 33, "puesto": "Machine Learning Engineer"},
]
print("Puestos con 'Data':", {p["nombre"]: p["puesto"] for p in personas if "Data" in
p["puesto"]})
```

Salida de comprensiones.py

```
Pares (con for): [2, 4, 6, 8, 10]
Pares (comprensión): [2, 4, 6, 8, 10]
Siguiente de cada par: [3, 5, 7, 9, 11]
Cuadrados de los múltiplos de 3: [9, 36, 81]
Puestos con 'Data': {'Alice': 'Data Scientist', 'Bob': 'Data Engineer'}
```

lambdas.py

```python
# lambda argumentos: expresión  (una función sin nombre, de una sola expresión)
cuadrado = lambda x: x ** 2
print(cuadrado(3))

cuadrado_mayor_a_diez = lambda x: x ** 2 > 10
print([cuadrado_mayor_a_diez(n) for n in (2, 3, 4, 5)])

# Las funciones son valores: se pueden guardar en variables y pasar como argumento
imprimir = print
imprimir("Hola desde una variable que guarda una función")
print(sorted(["Bermuda", "Aruba", "Key Largo"], key=lambda s: len(s)))
```

Salida de lambdas.py

```
9
[False, False, True, True]
Hola desde una variable que guarda una función
['Aruba', 'Bermuda', 'Key Largo']
```

map_filter_reduce.py

```python
from functools import reduce

lista = [1, 2, 3, 4, 5]

# map: aplica la función a cada elemento
print("Cuadrados:", list(map(lambda x: x ** 2, lista)))

# filter: se queda con los elementos para los que la función da True
numeros = list(range(-10, 11))
print("Negativos pares o positivos:", list(filter(lambda x: (x <= 0 and x % 2 == 0) or x > 0,
numeros)))

# reduce: acumula de a dos, de izquierda a derecha, hasta dejar un solo valor
print("Suma:", reduce(lambda acumulado, x: acumulado + x, lista))       # ((((1+2)+3)+4)+5)
print("Producto:", reduce(lambda acumulado, x: acumulado * x, lista))
```

Salida de map_filter_reduce.py

```
Cuadrados: [1, 4, 9, 16, 25]
Negativos pares o positivos: [-10, -8, -6, -4, -2, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Suma: 15
Producto: 120
```

ejercicio.py

```python
import random
from functools import reduce

random.seed(2026)        # así la salida es siempre la misma (sacalo para que cambie)

lst = ["Aruba", "Jamaica", "Bermuda", "Bahama", "Key Largo", "Montigo"]

# 01. Primer carácter de cada palabra (comprensión)
iniciales = [p[0] for p in lst]
print("01.", iniciales)

# 02. Longitud de cada palabra (comprensión)
longitudes = [len(p) for p in lst]
print("02.", longitudes)

# 03. Cantidad total de caracteres (reduce)
print("03.", reduce(lambda a, b: a + b, longitudes))

# Actividad: 50 enteros al azar entre 1 y 100 (comprensión)
numeros = [random.randint(1, 100) for _ in range(50)]
print("Números:", numeros)

# Filter: mayores a 50 y menores o iguales a 50
mayores = list(filter(lambda x: x > 50, numeros))
menores = list(filter(lambda x: x <= 50, numeros))
print("Mayores a 50:", mayores)
print("Menores o iguales a 50:", menores)

# Map: solo los impares, con el signo invertido (filter elige los impares, map los niega)
impares_negativos = list(map(lambda x: -x, filter(lambda x: x % 2 != 0, numeros)))
print("Impares con signo invertido:", impares_negativos)

# Reduce: sumatoria de la lista anterior
print("Sumatoria:", reduce(lambda a, b: a + b, impares_negativos))
```

Salida de ejercicio.py

```
01. ['A', 'J', 'B', 'B', 'K', 'M']
02. [5, 7, 7, 6, 9, 7]
03. 41
Números: [16, 41, 65, 66, 83, 14, 29, 77, 80, 72, 54, 74, 71, 94, 100, 99, 63, 97, 99, 76, 57,
31, 1, 79, 11, 15, 37, 13, 58, 2, 88, 63, 87, 41, 27, 51, 33, 45, 46, 49, 96, 66, 82, 10, 93, 44,
12, 72, 69, 38]
Mayores a 50: [65, 66, 83, 77, 80, 72, 54, 74, 71, 94, 100, 99, 63, 97, 99, 76, 57, 79, 58, 88,
63, 87, 51, 96, 66, 82, 93, 72, 69]
Menores o iguales a 50: [16, 41, 14, 29, 31, 1, 11, 15, 37, 13, 2, 41, 27, 33, 45, 46, 49, 10,
44, 12, 38]
Impares con signo invertido: [-41, -65, -83, -29, -77, -71, -99, -63, -97, -99, -57, -31, -1,
-79, -11, -15, -37, -13, -63, -87, -41, -27, -51, -33, -45, -49, -93, -69]
Sumatoria: -1526
```